PROJECT REPORT

Smart Product Pricing

Submitted in partial fulfilment of the requirements for

Machine Learning Lab- 20CP401P

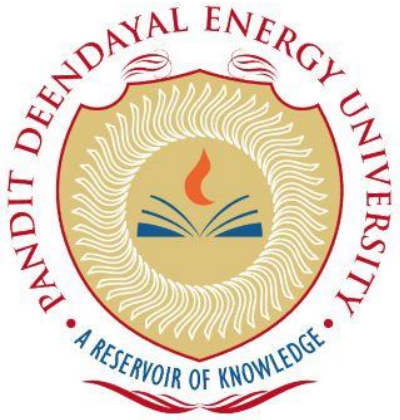
Submitted by:

Nisarg Mulani (23BCP224),

Mahir Patel (23BCP220)

Submitted to:

Dr. Sidheshwar Routray



Department of Computer Science & Engineering,

School of Technology, Pandit Deendayal Energy University,

Gandhinagar 382 426

INDEX

#	Topic	Pg No.
1.	Introduction and Background	3
2.	Problem Statement, Aim and Objectives	4
3.	Dataset Description	5
4.	Exploratory Data Analysis	6
5.	Data Preparation and Feature Engineering	9
6.	Methodology and Algorithms	11
7.	Experimental Setup and Evaluation Metrics	12
8.	Results and Model Comparison	13
9.	Interpretation and Discussion	14
10.	Limitations, Ethics and Future Work	15
11.	Conclusion and References	16

1. INTRODUCTION AND BACKGROUND

1.1 Introduction

Product pricing is an important aspect of e-commerce because it influences customer purchasing decisions, sales performance, and business profitability. Online marketplaces contain numerous products with different specifications, brands, quantities, and descriptions. Determining an appropriate price for each product can be challenging because product prices depend on multiple characteristics.

Traditional pricing methods often rely on manual analysis, predefined rules, or comparisons with similar products. These approaches may become inefficient when handling large product catalogues and may fail to capture complex relationships between product descriptions and their corresponding prices.

The **Smart Product Pricing Solution** is a machine learning project developed to predict product prices using textual product information. The project applies natural language processing (NLP), feature engineering, dimensionality reduction, and regression algorithms to learn patterns from existing product data and estimate prices for unseen products.

The proposed approach combines semantic text embeddings and TF-IDF features with multiple machine learning models. A stacking-based regression architecture combines the predictions of different models to generate the final price estimates.

1.2 Background

Product descriptions contain valuable information, including product types, specifications, materials, quantities, and other characteristics that may influence price. However, these descriptions are unstructured text and must be transformed into numerical representations before they can be used by most machine learning algorithms.

The project addresses this challenge through text preprocessing and feature extraction. Sentence Transformer embeddings represent semantic information, while TF-IDF captures statistical information about words. Principal Component Analysis (PCA) reduces the dimensionality of the TF-IDF features.

The resulting features are processed by XGBoost, LightGBM, CatBoost, and a Multi-Layer Perceptron (MLP). Their predictions are combined using a stacking regressor.

1.3 Motivation

The main motivation is to investigate how machine learning can automate product price estimation and reduce manual pricing effort.

The project aims to:

- Automate product price prediction using historical data.
- Extract useful information from unstructured product descriptions.
- Combine semantic and statistical text representations.
- Explore multiple regression algorithms for price estimation.
- Develop a reproducible pipeline that generates predictions for unseen products.

1.4 Project Scope

The project covers dataset inspection, exploratory data analysis, text preprocessing, feature extraction, regression model development, prediction generation, and output preparation.

The implemented pipeline focuses primarily on textual product information. Image-based feature extraction, live marketplace integration, and automated real-time repricing are outside the current implementation.

2. PROBLEM STATEMENT, AIM AND OBJECTIVES

2.1 Problem Statement

Predicting product prices is challenging because prices vary according to product characteristics and descriptions. Products with similar descriptions may have different prices, while differently described products may have comparable prices.

Manual price estimation is time-consuming and may not scale effectively to large product catalogues. Simple rule-based approaches may also fail to capture complex relationships between textual information and prices.

The problem addressed in this project is to develop a supervised machine learning system that learns from product descriptions and known prices to predict prices for previously unseen products.

2.2 Aim

The main aim is to design and implement a machine learning-based regression system that estimates product prices using textual product information and a combination of NLP and ensemble learning techniques.

2.3 Objectives

The major objectives are:

1. Analyze the product dataset and its price distribution.
2. Perform exploratory data analysis to identify patterns and extreme observations.
3. Clean and normalize textual product descriptions.
4. Generate semantic embeddings using Sentence Transformers.
5. Extract TF-IDF features and reduce their dimensionality using PCA.
6. Prepare the target price variable for regression.
7. Implement XGBoost, LightGBM, CatBoost, and MLP models.
8. Combine the models through a stacking regressor.
9. Generate predictions for the testing dataset.
10. Evaluate model performance and identify potential improvements.

2.4 Proposed Solution

The proposed solution begins by loading and inspecting the product dataset. Product descriptions are cleaned and transformed into semantic embeddings and TF-IDF features. PCA is applied to the TF-IDF representation, and the resulting features are combined and standardized.

The prepared data is passed to several regression algorithms. A stacking regressor combines their predictions to generate the final price estimates. These predictions are then converted back to the original price scale, where required, and exported into a CSV file.

3. DATASET DESCRIPTION

3.1 Dataset Overview

The training dataset inspected for this project contains **75,000 rows and 4 columns**. It includes product identifiers, textual descriptions, image links, and product prices.

The dataset is used to train a supervised regression model in which product information serves as the input and price is the target variable.

3.2 Dataset Attributes

Attribute	Data type	Description
sample_id	int64	Product record identifier
catalog_content	object	Textual product description
image_link	object	Link associated with the product image
price	float64	Actual product price

The primary model input is catalog_content. Although image_link is available in the dataset, the implemented pipeline primarily uses textual features rather than image-derived features.

3.3 Data Quality

The dataset inspection produced the following results:

Parameter	Result
Total rows	75,000
Total columns	4
Missing values in sample_id	0
Missing values in catalog_content	0
Missing values in image_link	0
Missing values in price	0

No missing values were observed in the inspected dataset. Therefore, missing-value imputation was not required for these columns based on this inspection.

3.4 Numerical Summary

Statistic	sample_id	price
Count	75,000	75,000
Mean	149,841.92	23.6477
Standard deviation	86,585.35	33.3769
Minimum	0	0.13
25th percentile	73,845.75	6.795
Median	150,129	14.00
75th percentile	225,040.25	28.625
Maximum	279,600*	2,796.00

The mean price is approximately 23.65, while the median is 14.00. The mean being higher than the median suggests a positively skewed price distribution. The large standard deviation and maximum price indicate substantial variation in the target variable.

These characteristics make price prediction challenging and motivate the use of suitable preprocessing and regression techniques.

3.5 Dataset Challenges

The primary challenges include unstructured text, variation in product descriptions, a wide range of prices, and potentially useful numerical details embedded in the text. Removing digits and punctuation during preprocessing may also eliminate information such as product dimensions and model numbers.

4. EXPLORATORY DATA ANALYSIS

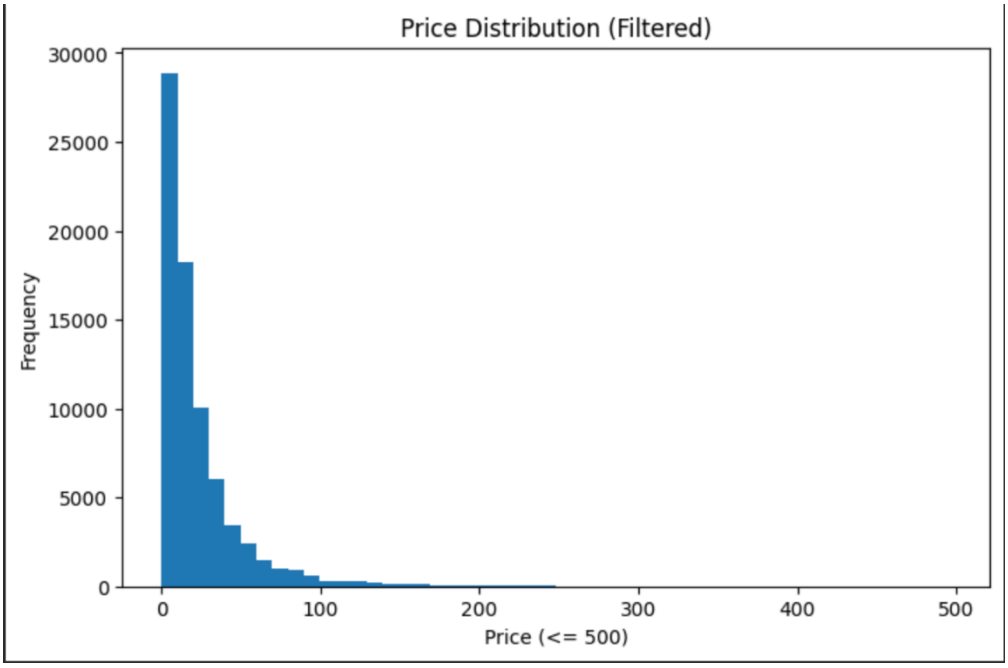
4.1 Purpose of EDA

Exploratory Data Analysis (EDA) is performed to understand the structure, distribution, and characteristics of the dataset before model training. It helps identify patterns, extreme values, and potential relationships between product information and prices.

4.2 Price Distribution

The price distribution shows substantial variation. The median price is 14.00, while the mean is approximately 23.65. The maximum observed price is 2,796.00.

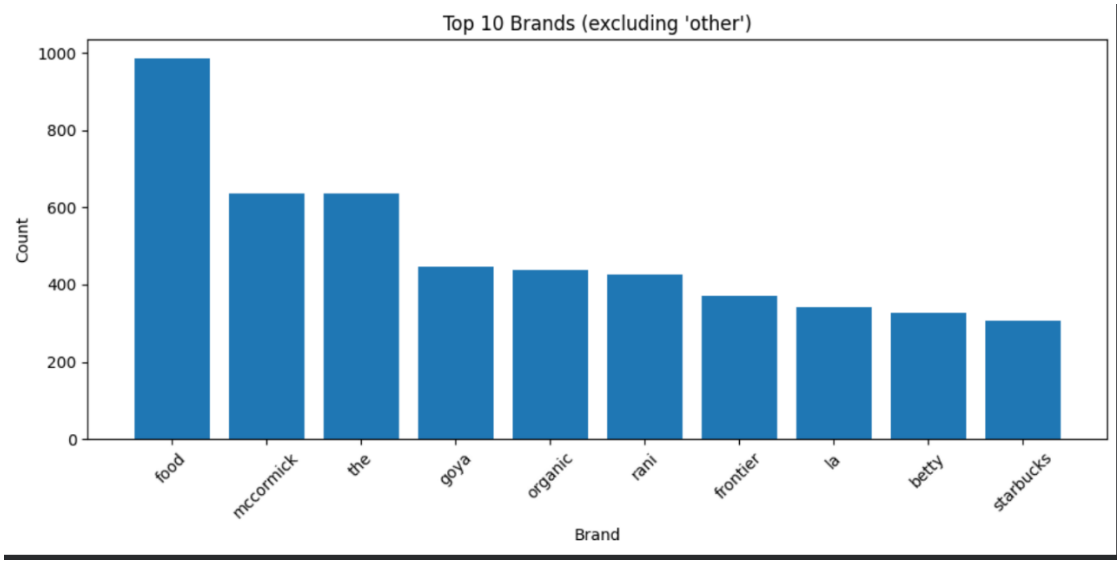
The difference between the mean and median suggests that high-priced products influence the overall distribution. Such extreme observations may affect regression errors and the model's ability to learn general price patterns.



4.3 Brand-Wise Analysis

Brand-related visualizations indicate variation in product prices and the presence of high-value outliers. The group labelled “other” also contains high-price observations, but this group may represent several brands rather than a single brand.

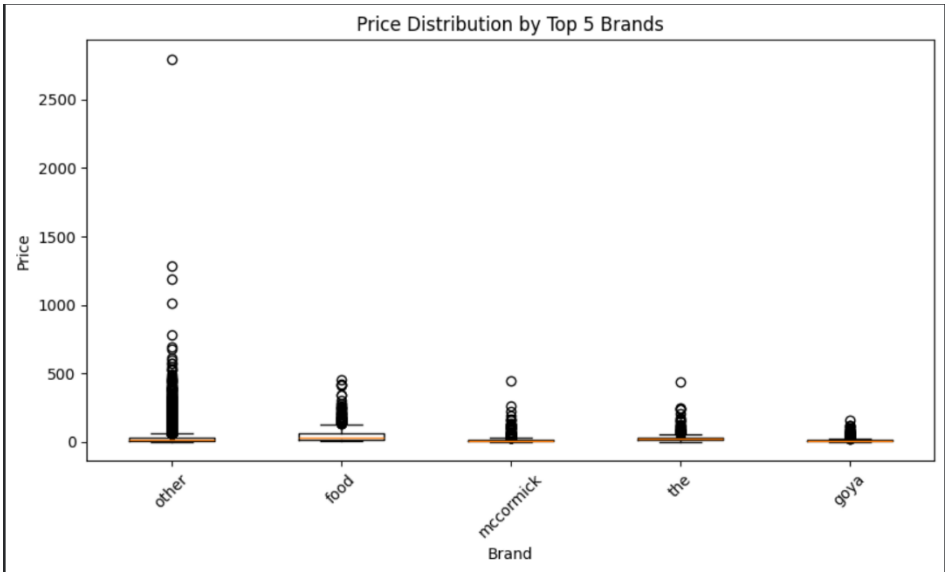
A separate brand-frequency chart shows the number of records associated with frequently occurring brands. These counts describe dataset representation and should not be interpreted as average brand prices.



4.4 Quantity and Price Analysis

The item-quantity-per-package analysis shows variation in average prices across quantity levels. Some quantity levels have relatively sparse observations, so their average prices may be unstable.

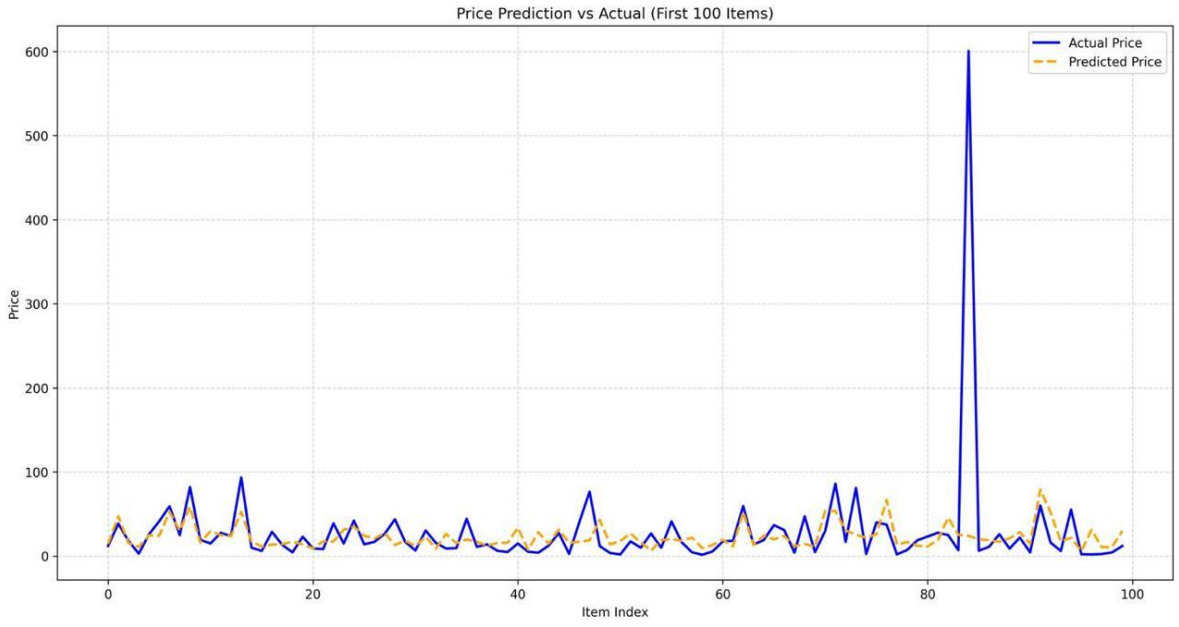
Quantity information may help explain pricing differences, but its usefulness in the final model depends on whether it is consistently extracted and included in the feature engineering pipeline.



4.5 Actual Versus Predicted Prices

The actual-versus-predicted visualization for the first 100 observations shows that predictions generally follow the lower-price region. A noticeable deviation occurs near observation index 84, where the model does not fully capture a sharp increase in the actual price.

This indicates that individual high-price observations may be difficult to predict. However, a plot of 100 observations cannot establish overall model performance; numerical evaluation metrics are also required.



4.6 Key Findings

The major observations from EDA are:

- The price distribution appears positively skewed.

- Product prices exhibit substantial variation.
- High-price observations are present.
- Some textual numerical features are strongly correlated.
- Brand frequency and brand-wise price variation provide useful descriptive context.
- Some individual predictions deviate substantially from actual prices.

These findings support the use of text feature engineering and regression modelling while highlighting the importance of careful evaluation.

5. DATA PREPARATION AND FEATURE ENGINEERING

5.1 Text Preprocessing

The catalog_content column is cleaned before feature extraction. The implemented procedure includes:

1. Converting text to lowercase.
2. Removing HTML-like tags.
3. Removing URLs.
4. Removing digits.
5. Removing punctuation.
6. Normalizing whitespace.

These operations standardize the descriptions and reduce unwanted textual elements. However, removing digits and punctuation can also eliminate useful product specifications, which is a limitation of this approach.

5.2 Sentence Transformer Embeddings

The project uses the pretrained all-MiniLM-L6-v2 Sentence Transformer model to generate dense numerical embeddings from product descriptions.

The embeddings provide semantic representations of the text and help the regression models process textual information in numerical form.

The process involves loading the pretrained model, supplying cleaned descriptions, generating embeddings, and preparing the resulting vectors for model training.

5.3 TF-IDF Feature Extraction

TF-IDF represents words numerically according to their occurrence within individual descriptions and across the dataset.

The implementation uses English stop-word handling and a maximum of 5,000 TF-IDF features. This representation captures statistical word-level information that complements the semantic embeddings.

5.4 Dimensionality Reduction

Principal Component Analysis is applied to the TF-IDF representation to reduce its dimensionality.

The implementation uses 50 components and a random state of 42. The reduced TF-IDF features are subsequently combined with the Sentence Transformer embeddings.

5.5 Feature Combination and Scaling

The project combines the semantic embeddings and PCA-transformed TF-IDF features into a single feature representation.

StandardScaler is applied to standardize the combined numerical features. This is especially relevant for the MLP model, whose optimization can be affected by differences in feature scales.

For reliable evaluation, preprocessing parameters should be learned from training data only and then applied to validation and testing data.

5.6 Target Transformation

The price variable is filtered using the 1st and 99th percentiles. This reduces the influence of extreme training prices but may also limit the model's ability to learn from products outside the retained range.

The target is transformed using:

$$y' = \log(1 + y)$$

y'= logarithmically transformed price

After prediction, the inverse transformation is applied:

$$\hat{y} = \exp(\hat{y}') - 1$$

$\hat{y}'$ = predicted price on the logarithmic scale

This transformation compresses large price values during training and restores predictions to the original price scale.

5.7 Feature Engineering Workflow

The feature engineering pipeline consists of the following stages:

1. Product description input.
2. Text cleaning and normalization.
3. Sentence Transformer embedding generation.
4. TF-IDF feature extraction.
5. PCA dimensionality reduction.
6. Feature combination and scaling.
7. Input to the regression models.
8. Price prediction and inverse target transformation.

This workflow converts unstructured product text into numerical representations suitable for supervised regression.

6. METHODOLOGY AND ALGORITHMS

6.1 Overall Methodology

The project follows a supervised machine learning methodology. Product descriptions are transformed into numerical features, and regression algorithms learn the relationship between those features and known product prices.

The overall process includes data loading, exploratory analysis, preprocessing, feature extraction, target preparation, model training, prediction generation, and result export.

6.2 XGBoost Regressor

XGBoost is a gradient boosting algorithm that builds an ensemble of decision trees sequentially. Each new tree helps improve the predictions produced by the existing ensemble.

The implementation uses the following principal parameters:

Parameter	Value
Number of estimators	600
Learning rate	0.05
Maximum depth	8
Subsample	0.8
Column subsample	0.8
Regularization parameter	1
Random state	42

XGBoost can learn nonlinear relationships between the extracted textual features and product prices.

6.3 LightGBM Regressor

LightGBM is a gradient boosting framework designed for efficient training on structured numerical features. It constructs decision trees to model relationships between input features and the target variable.

The implementation uses 600 estimators, a learning rate of 0.05, 64 leaves, feature fraction of 0.8, bagging fraction of 0.8, and random state 42.

LightGBM provides another tree-based learning approach that can capture patterns different from those learned by XGBoost.

6.4 CatBoost Regressor

CatBoost is a gradient boosting algorithm that builds an ensemble of decision trees. It is included as another regression model to learn nonlinear relationships in the engineered feature space.

The implementation uses 800 iterations, a learning rate of 0.05, depth 7, and random state 42.

6.5 Multi-Layer Perceptron

The Multi-Layer Perceptron is a feedforward neural network capable of learning nonlinear relationships through interconnected layers of neurons.

The implementation uses hidden layers containing 256, 128, and 64 neurons, with ReLU activation, the Adam optimizer, a learning rate of 0.001, a maximum of 600 iterations, and early stopping enabled.

Feature scaling is particularly important for this model because numerical feature magnitudes can affect optimization.

6.6 Stacking Regressor

Stacking is an ensemble learning technique that combines predictions from multiple base models to produce a final estimate.

In this project, XGBoost, LightGBM, CatBoost, and MLP are used as base regressors within the stacking approach.

The purpose of stacking is to combine different learning strategies. Tree-based models can capture nonlinear feature interactions, while the MLP can learn patterns through neural network layers.

The combined model aims to use the strengths of these approaches to generate price predictions. However, the actual benefit of stacking must be established through a controlled comparison against individual models.

7. EXPERIMENTAL SETUP AND EVALUATION METRICS

7.1 Experimental Environment

The project is implemented in Python using a Jupyter Notebook workflow.

The principal tools and libraries include:

Component	Technology
Programming language	Python
Development environment	Jupyter Notebook
Data manipulation	Pandas, NumPy
Text representation	Sentence Transformers, TF-IDF
Dimensionality reduction	Scikit-learn PCA
Feature scaling	Scikit-learn StandardScaler
Regression models	XGBoost, LightGBM, CatBoost, MLP
Output format	CSV

7.2 Data Preparation

The dataset is loaded from CSV files. Product descriptions are cleaned and converted into numerical features, while prices are prepared for regression through percentile filtering and logarithmic transformation.

A key consideration is ensuring that feature extraction and scaling are performed without using information from validation records. If preprocessing is fitted before splitting the data, information from the validation subset may influence the evaluation.

7.3 Validation Approach

The notebook includes a RepeatedKFold configuration with five splits and two repeats. However, the implementation uses next(iter(...)), which retrieves only the first generated split rather than executing all repeated folds.

Therefore, the presence of a repeated cross-validation configuration should not be interpreted as a completed evaluation across all ten folds. A complete evaluation would require training and evaluating the model across every intended split.

7.4 Evaluation Metric: Root Mean Squared Error

Root Mean Squared Error (RMSE) measures the typical magnitude of prediction errors while giving greater weight to larger errors.

It is calculated as:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Where:

- (n) is the number of evaluated observations.
- (y_i) is the actual price.
- ($\hat{y}_i$) is the predicted price.

A lower RMSE indicates smaller squared prediction errors on the evaluated dataset. RMSE is expressed in the same units as the original target variable when calculated after inverse transformation.

8. RESULTS AND MODEL COMPARISON

8.1 Prediction Output

The project generates price predictions for records in the testing dataset. The predictions are exported to Predicted-prize/test_out.csv.

The output contains a product identifier and the corresponding predicted price, allowing each prediction to be associated with its original record.

The prediction file represents the final output of the implemented machine learning pipeline.

8.2 Reported Model Performance

The developed Smart Product Pricing Solution achieved an RMSE error of approximately 27%. This result indicates the prediction error obtained during model evaluation. The objective of the model is to minimize this error and improve the accuracy of product price predictions.

8.3 Model Comparison

The implemented models use different learning strategies:

Model	Learning approach	Purpose
XGBoost	Gradient-boosted decision trees	Learn nonlinear feature relationships
LightGBM	Efficient gradient boosting	Capture patterns using tree ensembles
CatBoost	Gradient-boosted decision trees	Provide an alternative boosting approach
MLP	Feedforward neural network	Learn nonlinear relationships through neural layers
Stacking regressor	Ensemble of base regressors	Combine predictions from different models

The project incorporates these models into a stacking-based prediction pipeline. However, a complete numerical comparison of the individual models cannot be established without separate, verified evaluation results for each model.

8.4 Result Summary

The implemented pipeline demonstrates how semantic embeddings, TF-IDF features, dimensionality reduction, and ensemble regression can be combined to predict product prices.

The reported RMSE provides an initial indication of prediction performance, while the actual-versus-predicted visualization highlights that some individual price variations remain difficult to capture.

A stronger performance assessment would require leakage-free validation, evaluation across all intended folds, and a comparison of individual regressors against the stacking model.

9. INTERPRETATION AND DISCUSSION

9.1 Interpretation of the Approach

The Smart Product Pricing Solution combines semantic and statistical representations of product descriptions.

Sentence Transformer embeddings represent semantic information, while TF-IDF captures word-level patterns. Their combination provides the regression models with different forms of information extracted from the same textual input.

PCA reduces the dimensionality of the TF-IDF features, and feature scaling prepares the combined numerical representation for model training.

9.2 Role of Ensemble Learning

The project combines XGBoost, LightGBM, CatBoost, and MLP within a stacking-based approach.

These algorithms learn relationships in different ways. Tree-based models construct nonlinear decision boundaries through boosting, whereas MLP learns patterns through interconnected neural network layers.

Stacking provides a mechanism for combining their predictions. Nevertheless, an ensemble is not automatically superior to every individual model; its effectiveness must be established using consistent validation data and evaluation metrics.

9.3 Challenges Observed

The principal challenges include:

- Handling unstructured product descriptions.
- Managing the wide range of product prices.
- Preserving useful information during text cleaning.
- Controlling the dimensionality of textual features.
- Avoiding data leakage during preprocessing and evaluation.
- Predicting unusually high-priced products accurately.

These challenges illustrate that model architecture alone does not determine prediction quality. Dataset preparation and evaluation methodology are equally important.

9.4 Practical Significance

The project demonstrates a possible application of machine learning in e-commerce operations. A reliable price prediction model could assist with initial price estimation, catalog analysis, and identification of unusual pricing patterns.

However, predicted prices should not automatically be treated as optimal selling prices. Commercial pricing decisions may also require market demand, competitor prices, product availability, taxes, shipping costs, and business objectives.

10. LIMITATIONS, ETHICS AND FUTURE WORK

10.1 Limitations

The current implementation has several limitations.

Loss of textual information: Removing digits and punctuation may eliminate important product specifications, including quantities, dimensions, and model numbers.

Extreme-value filtering: Filtering training prices using percentile thresholds changes the training distribution and may reduce the ability to predict prices outside the retained range.

Potential data leakage: If preprocessing is fitted before the training-validation split, information from validation records may influence the learned feature transformations.

Incomplete cross-validation execution: The configured repeated cross-validation process retrieves only the first split rather than executing every intended fold.

Limited numerical comparison: Separate, verified evaluation scores for all base models are not available in the reported results.

Text-focused modelling: Although image links are present, image-derived features are not part of the implemented prediction pipeline.

Generalization uncertainty: The reported RMSE does not, by itself, establish performance on all unseen product types or price ranges.

10.2 Ethical Considerations

Price prediction models should be used carefully because inaccurate estimates may lead to unsuitable pricing decisions.

The quality of predictions depends on the representativeness and reliability of the training data. If certain product types or price ranges are underrepresented, the model may perform poorly on those products.

Predictions should therefore be treated as estimates rather than guaranteed market prices. Human review and additional business information remain important for consequential pricing decisions.

10.3 Future Work

Potential improvements include:

1. Preserve relevant numbers and product specifications during text preprocessing.
2. Fit preprocessing transformations only on training data and apply them to validation and testing data.
3. Execute complete cross-validation across all intended folds.
4. Compare each base model against the stacking regressor using identical evaluation splits.
5. Report additional regression metrics, such as MAE and (R^2), alongside RMSE.
6. Investigate feature engineering for product quantity, dimensions, and brand information.
7. Explore image-based features from product photographs where appropriate.
8. Tune model hyperparameters using a reliable validation strategy.
9. Evaluate prediction errors separately across different price ranges.
10. Investigate deployment options after model performance and reliability have been established.

11. CONCLUSION AND REFERENCES

11.1 Conclusion

The Smart Product Pricing Solution demonstrates the application of natural language processing and machine learning to product price prediction.

The project processes textual product descriptions, generates Sentence Transformer embeddings, extracts TF-IDF features, applies PCA, and combines the resulting representations for regression modelling. XGBoost, LightGBM, CatBoost, and MLP are incorporated into a stacking-based approach intended to combine different learning strategies.

The project also includes dataset inspection, exploratory data analysis, target transformation, prediction generation, and CSV output preparation. The reported RMSE of approximately 27 provides an initial performance result, although further controlled evaluation is required to establish generalization performance reliably.

The implementation highlights the importance of text preprocessing, feature representation, ensemble learning, and leakage-free evaluation in developing a machine learning solution.

Overall, the project provides practical experience in building a complete machine learning pipeline for an e-commerce pricing problem and identifies several opportunities for improving model reliability and prediction quality.

11.2 References

1. Reimers, N., and Gurevych, I. (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." *Proceedings of EMNLP-IJCNLP*. https://doi.org/10.18653/v1/D19-1410
2. Salton, G., and Buckley, C. (1988). "Term-Weighting Approaches in Automatic Text Retrieval." *Information Processing & Management*, 24(5), 513–523. https://doi.org/10.1016/0306-4573(88)90021-0
3. Chen, T., and Guestrin, C. (2016). "XGBoost: A Scalable Tree Boosting System." *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. https://doi.org/10.1145/2939672.2939785
4. Ke, G., et al. (2017). "LightGBM: A Highly Efficient Gradient Boosting Decision Tree." *Advances in Neural Information Processing Systems*, 30. https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html
5. Prokhorenkova, L., et al. (2018). "CatBoost: Unbiased Boosting with Categorical Features." *Advances in Neural Information Processing Systems*, 31. https://proceedings.neurips.cc/paper_files/paper/2018/hash/14491b756b3a51daac41c24863285549-Abstract.html
6. Wolpert, D. H. (1992). "Stacked Generalization." *Neural Networks*, 5(2), 241–259. https://doi.org/10.1016/S0893-6080(05)80023-1
7. Scikit-learn. "StackingRegressor Documentation." https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.StackingRegressor.html
8. Hugging Face. "all-MiniLM-L6-v2 Sentence Transformer Model." https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2